Day 21 (Mon Nov 9): Exceptions and try/except

Smooth | Python | closed Nov 11 | covers only what was really done

Key ideas

- An unhandled error stops the program with a traceback; the lines after it never run.
- try/except ZeroDivisionError: the except name must match the raised error, or the error is not caught.
- After the first error inside try, Python jumps to the matching except and skips the rest of the try.
- ValueError = right type, bad content (int("abc"), int("12.5"), int("")). TypeError = wrong type for the operation (int(None), 10 + "a").
- except ValueError as e gives the message in e; e is deleted after the except block.
- Bare except and except Exception swallow every error and hide bugs. Catch the narrowest error you can name.
- An error that is not caught leaves the function and stops the program. That is a bug you can see.

Examples (run and verified)

```
try:
    n = int("12.5")
except ValueError as e:
    print("Bad:", e)
# prints: Bad: invalid literal for int() with base 10: '12.5'

try:
    print(10 / 0)
except ValueError:
    print("never reached")
# raises ZeroDivisionError: except name does not match, program stops
```

int(None) raises TypeError: int() argument must be a string, a bytes-like object or a real number, not 'NoneType'.

Interview edge

- except Exception is almost a bare except. The caller cannot tell "b was 0" from "b was a string", so a real bug hides behind None.
- In an if/elif chain each elif already knows the earlier conditions failed. Use ">= lower bound"; a range like "<= 74" leaves a gap for decimals.
- A function that returns a value should not also print. The caller decides what to show.

My mistakes and the fix

- Answered "A" for 74.5 in a broken chain: I skipped the first check. Fix: write every check in order, left to right.
- Wrote except Exception where the spec said only b = 0. Fix: name the exact error the spec asks for.
- Answered for my old code, not the version asked. Fix: re-read the question and the code before answering.
- Reasons were thin ("75 equals 75"). Fix: write the check with values, e.g. 75 >= 75 is True, so it returns.

How I improved

- Functions recall (grade_of, no return gives None, None + 10 raises TypeError) was correct from memory.
- I diagnosed my own error ("I misread the first condition") and explained why TypeError is not caught in my own words.
- Traced 74 and 74.0001 through both checks without help.

Practice to retry (no solutions)

- Write safe_divide(a, b) from a blank file: returns a / b, returns None only when b is 0. Narrow except, no print inside. Give the result of safe_divide(10, 2), safe_divide(10, 0), safe_divide(10, "a") with reasons; say returns or raises.
- Rewrite grade_of with the elif written as "mark <= 74". Trace 75, 74.5, 74, 74.0001 through every check.
- Write a function to_int(text) that returns the integer, or None if the text is not a number. Which error do you catch? What do int("7"), int("7.5"), int("") give?
- Which error does each raise: int("abc"), int(None), 5 + "5", 5 / 0, [1, 2][5]? Check by running.

Recall questions (answer from memory)

- What happens to an unhandled error, and why must the except name match?
- ValueError vs TypeError: one example each.
- What is wrong with a bare except or except Exception?
- What happens to the rest of the try block after the first error?
- Why is e not usable after the except block?

Open items (carried to Day 22)

- else clause, FileNotFoundError, modules (import, math, random, datetime).
- Practice 2: ask until a valid number (while loop, input, try/except ValueError).
- Practice 3: days left to Dec 31 with datetime.
- README and .gitignore for one project.
- Split grade_counts into a grade_of helper. Retest safe_divide.